

AFRILANG Stdlib

std/c/statrmse

Bibliothèque AFRILANG `std/c/statrmse` (niveau complex, catégorie complex). Elle expose 6 fonction(s) :
rmse, mse, mae,

```
`import "std/c/statrmse" . `use statrmse`
```

- `rmse(actual list number, predicted list number) ? number`
- `mse(actual list number, predicted list number) ? number`
- `mae(actual list number, predicted list number) ? number`
- `nrmse(actual list number, predicted list number) ? number`
- `smape(actual list number, predicted list number) ? number`
- `r2Score(actual list number, predicted list number) ? number`

Category: complex | Tier: complex | Functions: 6

FRANCAIS

1. Vue d'ensemble

Bibliothèque AFRILANG `std/c/statrmse` (niveau complex, catégorie complex). Elle expose 6 fonction(s) : rmse, mse, mae,

```
`import "std/c/statrmse"` . `use statrmse`  
- `rmse(actual list number, predicted list number) ? number`  
- `mse(actual list number, predicted list number) ? number`  
- `mae(actual list number, predicted list number) ? number`  
- `nrmse(actual list number, predicted list number) ? number`  
- `smape(actual list number, predicted list number) ? number`  
- `r2Score(actual list number, predicted list number) ? number`
```

2. Installation & import

Ajoutez le module à votre programme :

```
import "std/c/statrmse"  
use statrmse
```

Niveau : complex · Catégorie : Modules complex (c/)

Domaines avancés ? graphes, NLP, réseaux complexes.

3. Référence API

```
? rmse(actual list number, predicted list number) ? number  
? mse(actual list number, predicted list number) ? number  
? mae(actual list number, predicted list number) ? number  
? nrmse(actual list number, predicted list number) ? number  
? smape(actual list number, predicted list number) ? number  
? r2Score(actual list number, predicted list number) ? number
```

4. Exemples d'utilisation

```
import "std/c/statrmse"  
use statrmse  
  
create result = rmse(/* args */)   
say result  
  
create result = mse(/* args */)   
say result  
  
create result = mae(/* args */)   
say result  
  
create result = nrmse(/* args */)   
say result  
  
create result = smape(/* args */)   
say result  
  
create result = r2Score(/* args */)   
say result
```

5. Bonnes pratiques

? Préférez `import "std/c/statrmse"` en tête de fichier.
? Lancez `afrilang check` avant de livrer.
? Combinez avec les modules stdlib de la même catégorie.
? Voir /stdlib/ pour le source live et les démos playground.
? Signalez les problèmes sur GitHub si une export manque.

6. Annexe ? source

```
module statrmse  
  
function rmse(actual list number, predicted list number) returns number  
end
```

```
function mse(actual list number, predicted list number) returns number
end

function mae(actual list number, predicted list number) returns number
end

function nrmse(actual list number, predicted list number) returns number
end

function smape(actual list number, predicted list number) returns number
end

function r2Score(actual list number, predicted list number) returns number
end
end
```

ENGLISH

1. Overview

AFRILANG library `std/c/statrmse` (tier complex, category complex). Exports 6 function(s): rmse, mse, mae, nrmse, smape,

```
`import "std/c/statrmse"` . `use statrmse`  
- `rmse(actual list number, predicted list number) ? number`  
- `mse(actual list number, predicted list number) ? number`  
- `mae(actual list number, predicted list number) ? number`  
- `nrmse(actual list number, predicted list number) ? number`  
- `smape(actual list number, predicted list number) ? number`  
- `r2Score(actual list number, predicted list number) ? number`
```

2. Installation & import

Add the module to your program:

```
import "std/c/statrmse"  
use statrmse
```

Tier: complex · Category: Complex modules (c/)
Advanced domains ? graphs, NLP, complex networks.

3. API reference

```
? rmse(actual list number, predicted list number) ? number  
? mse(actual list number, predicted list number) ? number  
? mae(actual list number, predicted list number) ? number  
? nrmse(actual list number, predicted list number) ? number  
? smape(actual list number, predicted list number) ? number  
? r2Score(actual list number, predicted list number) ? number
```

4. Usage examples

```
import "std/c/statrmse"  
use statrmse  
  
create result = rmse(/* args */)   
say result  
  
create result = mse(/* args */)   
say result  
  
create result = mae(/* args */)   
say result  
  
create result = nrmse(/* args */)   
say result  
  
create result = smape(/* args */)   
say result  
  
create result = r2Score(/* args */)   
say result
```

5. Best practices

```
? Prefer `import "std/c/statrmse"` at the top of your file.  
? Run `afirang check` before shipping.  
? Combine with related stdlib modules from the same category.  
? See /stdlib/ for the live source and playground demos.  
? Report issues on GitHub if an export is missing or undocumented.
```

6. Appendix ? source

```
module statrmse  
  
function rmse(actual list number, predicted list number) returns number  
end
```

```
function mse(actual list number, predicted list number) returns number
end

function mae(actual list number, predicted list number) returns number
end

function nrmse(actual list number, predicted list number) returns number
end

function smape(actual list number, predicted list number) returns number
end

function r2Score(actual list number, predicted list number) returns number
end
end
```